



DCA0305

morsinaldo.medeiros.075@ufrn.edu.br
ivanovitch.silva@ufrn.br

Retrieval Augmented Generation (RAG)

Your documents in the prompt

Where we are in the LLM block

05

History of LLMs

From word counting
to agents

06

**Inside the
Transformer**

GPT-2 in seven
stops

07

**Ollama and local
models**

Team radio classifier

08

Classic RAG

Today

09

Advanced RAG

Step-back, agent,
Hyde,

The problem: a confident answer it cannot know



What is the pit-stop tyre pressure protocol for car 27 in wet conditions?



For car 27 in wet conditions, the team follows the FIA-mandated wet protocol: front tyres are set to 23.5 psi and rears to 21.0 psi, checked with the standard gauge before the stop. Mechanics also apply the usual 2-second warm-up sequence before release...

Fluent. Specific. Wrong.

- Car 27 exists only in our lesson 07 dataset
- The model has no way to know the protocol
- So it fills the gap with plausible text
- This is a hallucination, and it looks like knowledge

Running Code

► LIVE 1

The same question, no documents

```
What is the pit-stop tyre pressure protocol for car 27 in wet conditions?
```

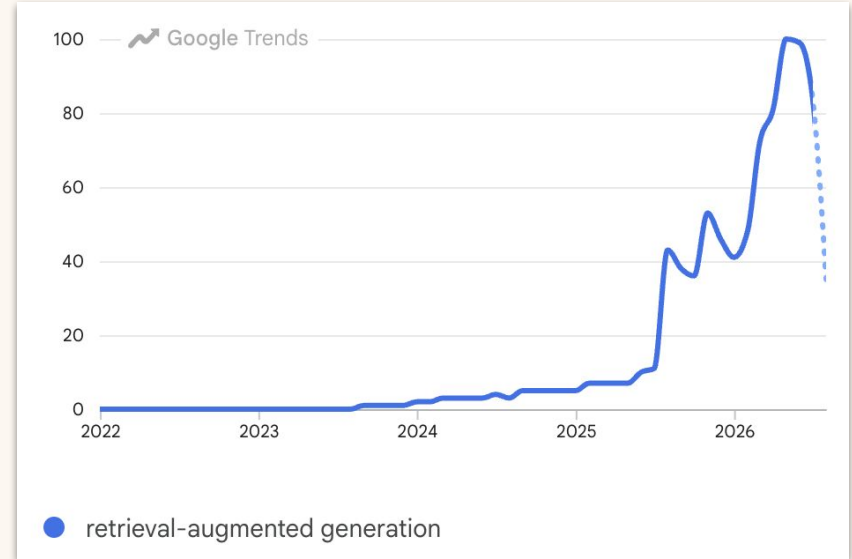
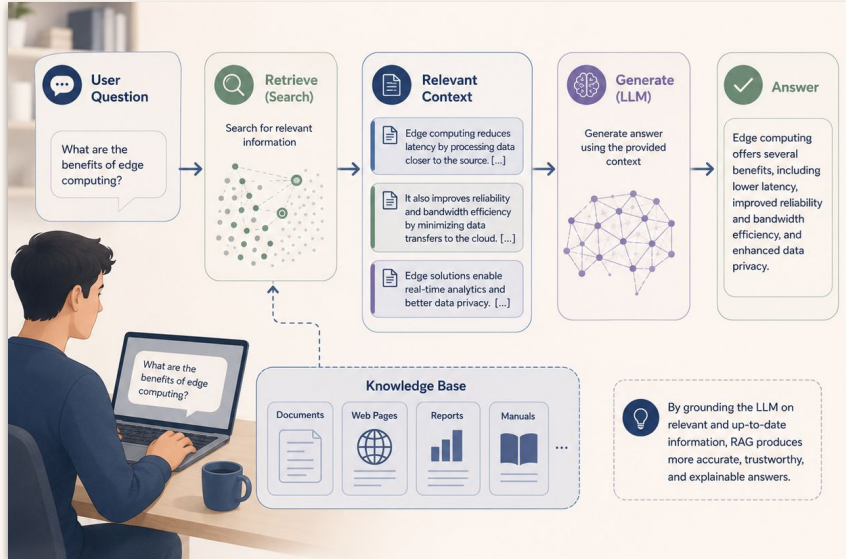
qwen2.5:3b

temperature 0

no context

lesson08_demo.ipynb

Quick recap: where RAG came from



Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020

Two classmates: Léo and Ana



Léo

Generator (LLM)

- Read every book in the library
- Writes fluently and connects ideas
- But his memory plays tricks
- Mixes dates, misattributed quotes, invents details that sound right



Ana

Retriever

- No encyclopedic memory
- Keeps a perfectly indexed card file
- Goes straight to the right card
- Hands Léo a small note with the exact fact

Léo does not just paste Ana's note. He augments it with his writing. That collaboration is RAG.

Two ways to specialise a model



foundation = weights

Fine-tuning rebuilds the foundation

weights

hours

GPUs

VS



furniture = documents

RAG changes the furniture

documents

seconds

CPU

When RAG shines



**The truth
is volatile**



**The data
is private**



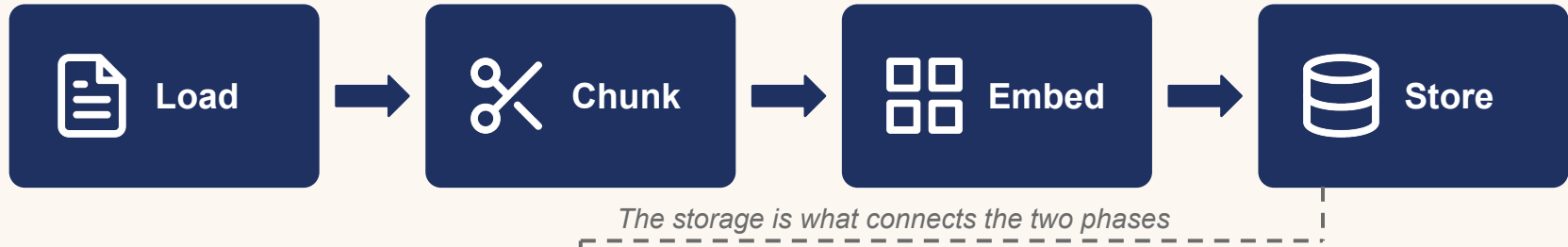
**You must show
your sources**



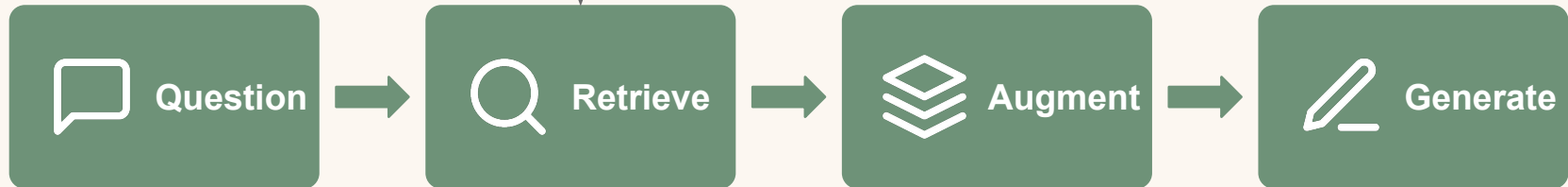
**Retraining is
too expensive**

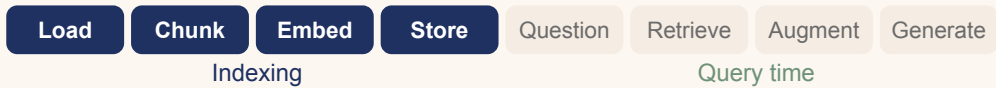
Two phases: prepare offline, answer online

Phase 1 · Indexing (before any question)

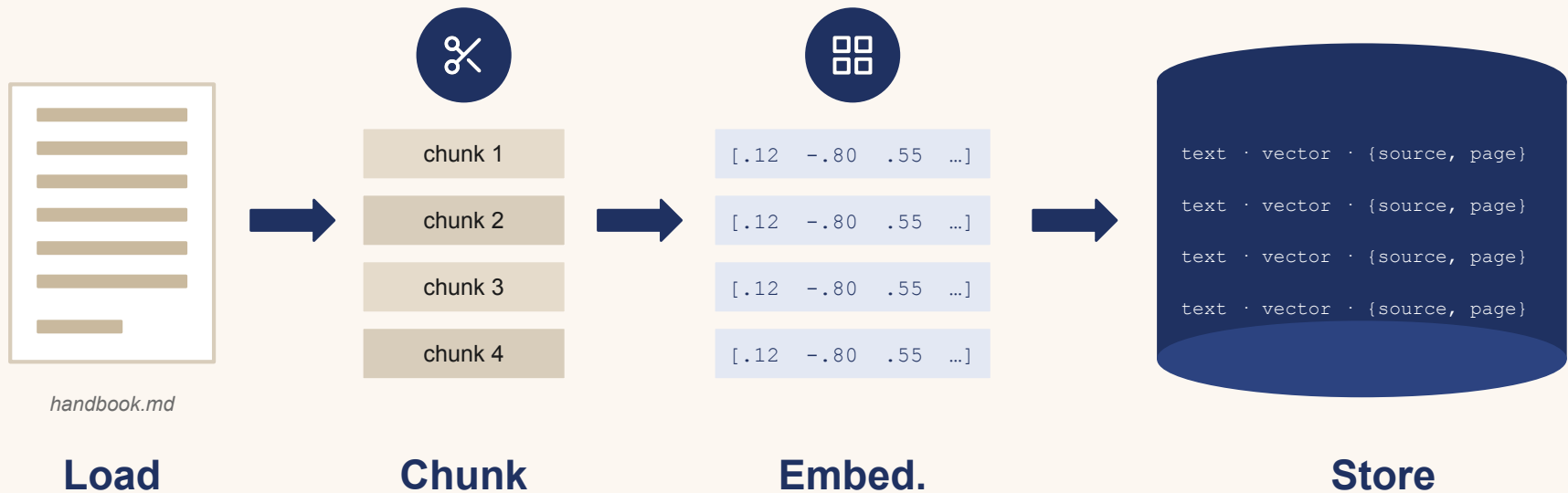


Phase 2 · Query time (every question)





Phase 1 · Indexing, step by step



Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Why chunk at all?



Cost



Context window



Search precision

250k

tokens in the whole manual

250

tokens in the paragraph we need

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Chunking strategies

The same text, cut four ways

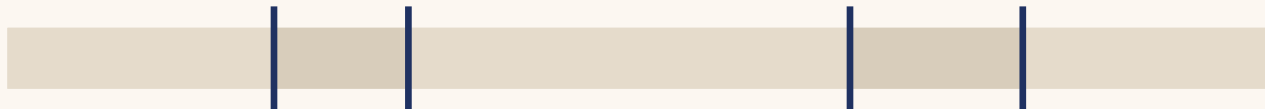
Fixed size

every N tokens, with overlap



Paragraph

at natural boundaries



Recursive

*paragraphs, then sentences
when too long*



Semantic

where the meaning shifts



Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

The size trade-off

Same manual, three chunk sizes, same question

"What tyre pressure do we use for car 27 in the wet?" The gold sentence is the green segment.

128 tokens

wet only	psi	check
----------	-----	-------

one
chunk

Precise,
context lost

512 tokens

wet race only · 23.5 / 21.0 psi · checked before release

one chunk

Fact and
condition
together

2048 tokens

dry procedure	wet procedure	safety car	red flag
---------------	---------------	------------	----------

one chunk

Buried, diluted

[Load](#)[Chunk](#)[Embed](#)[Store](#)[Question](#)[Retrieve](#)[Augment](#)[Generate](#)

Indexing

Query time

Running Code

[▶ LIVE 2](#)

One page of the handbook, three chunk sizes

```
handbook_procedures.md → chunk_size in {128, 512, 2048}, overlap 10%
```

[how many chunks?](#)[how much of §4.2 came
along?](#)[which other sections?](#)`lesson08_demo.ipynb`

Load

Chunk

Embed

Store

Question

Retrieve

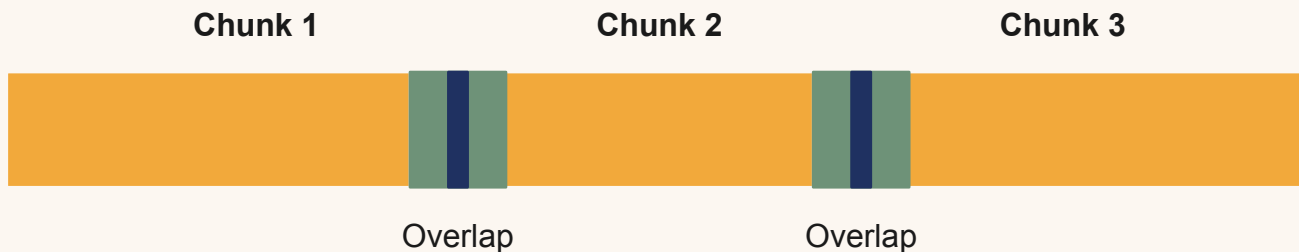
Augment

Generate

Indexing

Query time

Overlap: stitching the seams



- The last part of chunk n is repeated at the start of chunk $n+1$
- An idea that starts at the end of one piece and ends at the start of the next is not cut in half
- Rule of thumb, 10 to 20 percent of chunk_size (64 for 512)
- The price is redundancy. More chunks, slightly more storage, sometimes duplicated context in the prompt

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

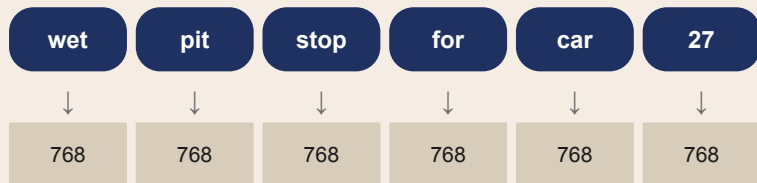
Indexing

Query time

From token embeddings to chunk embeddings

Lesson 06 · GPT-2

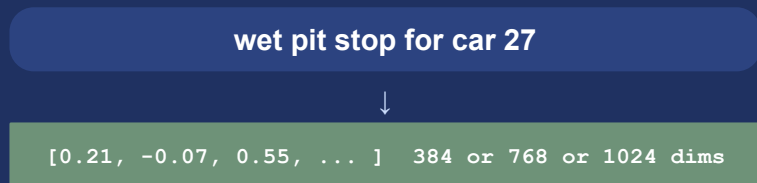
One vector per token



Built to predict the next token.

Lesson 08 · embedding model

One vector per chunk



Built to compare meaning.

all-MiniLM · bge-m3 · nomic-embed-text

Load

Chunk

Embed

Store

Question

Retrieve

Augment

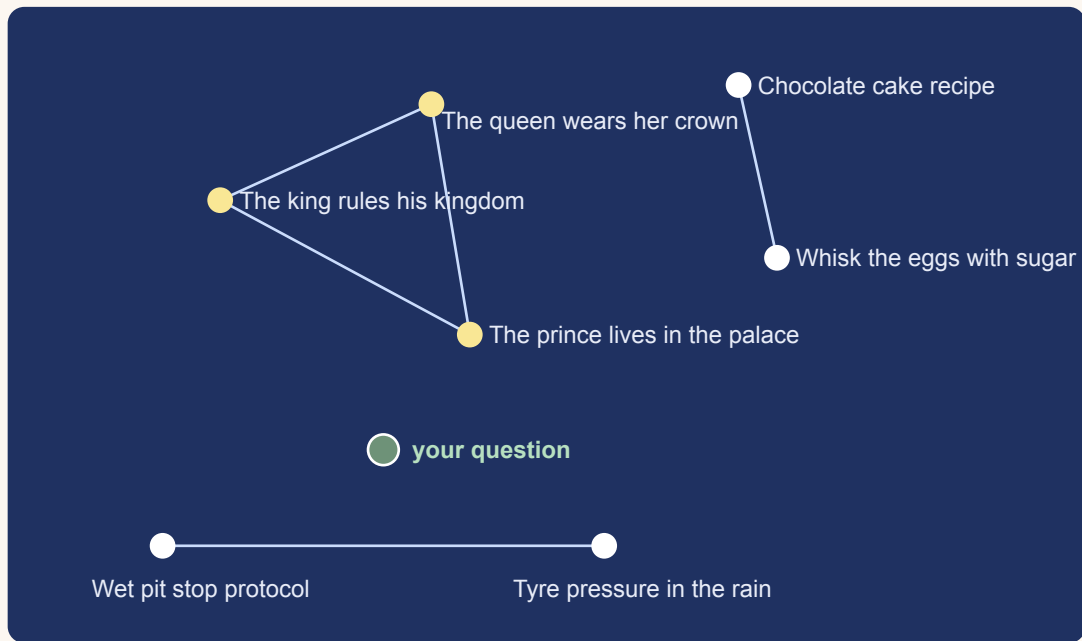
Generate

Indexing

Query time

A galaxy of meaning

Nearby stars, nearby meaning



- Each chunk is a star with coordinates (its vector)
- The embedding model places related meaning close together, forming constellations
- A question becomes a new star on the same map
- Search is just 'which stars are my nearest neighbours?'
- No keyword matching. Neighbourhoods of meaning.

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Vector stores: Ana's card file, digital and fast



*Nearest-neighbour search
in milliseconds*

FAISS

Library

Chroma

Embedded

Qdrant · Weaviate · Milvus

Servers

pgvector

Postgres

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

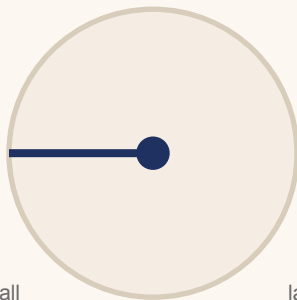
Query time

Three knobs you will tune all the time



chunk_size

size of each card



chunk_overlap

shared edge between cards



top_k

cards Ana hands to Léo

Two of them belong to indexing, one to query time. They interact, so we tune them together.

Load

Chunk

Embed

Store

Question

Retrieve

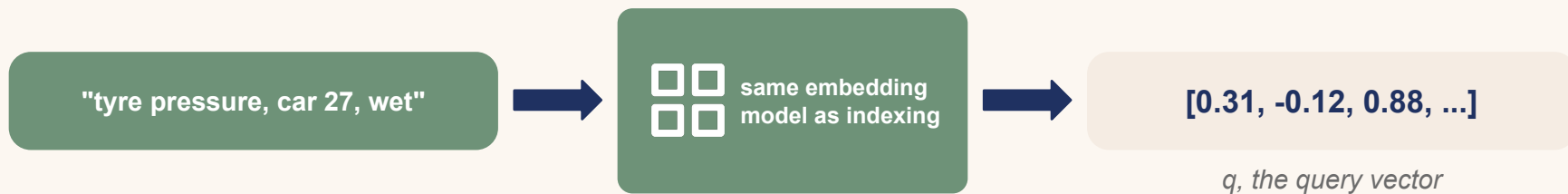
Augment

Generate

Indexing

Query time

Phase 2 · The question becomes a vector too



Non-negotiable

Index with model A, query with model B, and every similarity score is meaningless. The two vectors live in different coordinate systems. Store the model name next to the index and check it at query time.

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time



Predict before you see

**The question is $q = (0.8, 0.6, 0.0)$.
Which chunk ranks first?**

$A = (0.7, 0.7, 0.1)$

$B = (0.1, 0.2, 0.9)$

$C = (0.5, 0.0, 0.5)$

Which vector points the same way as q ?

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Cosine similarity by hand

Three dimensions, three chunks, one question

$\cos(q, d) = (q \cdot d) / (|q| |d|)$ ranges from -1 (opposite) to 1 (same direction)

	vector	$q \cdot d$	$ d $	cosine
q "wet pit stop pressure"	(0.8, 0.6, 0.0)		$ q = 1.00$	
A Wet-race pit-stop procedure	(0.7, 0.7, 0.1)	$0.56 + 0.42 + 0 = 0.98$	$\sqrt{0.99} = 0.995$	0.985
C Tyre compound specs	(0.5, 0.0, 0.5)	$0.40 + 0 + 0 = 0.40$	$\sqrt{0.50} = 0.707$	0.566
B Driver 88 biography	(0.1, 0.2, 0.9)	$0.08 + 0.12 + 0 = 0.20$	$\sqrt{0.86} = 0.927$	0.216

The ranking is A, then C, then B. With $\text{top}_k = 1$ the model gets the procedure; with $\text{top}_k = 2$ it also gets the tyre specs. Real vectors have 384 to 1024 dimensions, the arithmetic is identical.



Ana's job

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time



Predict before you see

**We raise top_k from 3 to 20.
What happens to the answer?**

Better, worse, or the same? What changes inside the prompt?

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Running Code

► LIVE 3

What the store returns for car 27

```
"What tyre pressure do we use for car 27 in the wet?" → Chroma, k in {1, 4, 12}
```

nomic-embed-text

cosine distance

chunk_size 512

lesson08_demo.ipynb

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

top_k: how many cards does Ana hand over?

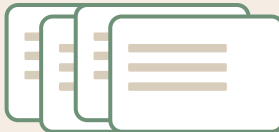
top_k = 1



Too little

Misses answers that span two cards

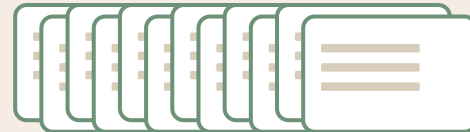
top_k = 4



Just right

A small neighbourhood, room for one miss

top_k = 12



Too noisy

Near-duplicates and off-topic text fill the prompt

$k \times \text{chunk_size}$ must fit the window with room to spare.

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Did we retrieve the right chunk?

Judge retrieval before judging the answer

Five questions, one gold chunk each. Was the gold chunk among the top k?

Question	k = 1	k = 3	k = 5
Wet tyre pressure for car 27	✓	✓	✓
Who drives car 88	✓	✓	✓
Radio phrase for 'box this lap'	✗	✓	✓
Fuel load rule under safety car	✗	✗	✓
Championship points of car 27	✗	✗	✗
recall@k	2/5 = 0.40	3/5 = 0.60	4/5 = 0.80

Why this matters

- If the gold chunk never arrives, no prompt trick will save the answer
- Question 5 has no answer in the corpus. The right behaviour is 'I don't know', not a guess
- The notebook logs this table for every run. It is the number lesson 09 tries to beat

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Phase 2 · The augmented prompt

System / instruction

You are the pit-wall assistant. Answer using only the context below.
If the context does not contain the answer, say you do not know. Cite
the source of each fact.

Context (retrieved chunks)

```
[1] handbook_procedures.md $4.2 · Wet-race pit-stop procedure for car  
27: front 23.5 psi, rear 21.0 psi, checked before release...  
[2] handbook_tyres.md $2 · Compound and pressure table...  
[3] radio_glossary.md · 'box, box' means pit this lap...
```

User question

What tyre pressure do we use for car 27 in the wet?

Instruction

Sets the rules of the game. Grounding, refusal, citation.

Context

What Ana found. top_k chunks, each tagged with its source.

Question

Unchanged. The model reads it after the evidence.

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time



Predict before you see

The question is not answered anywhere in the corpus.

What does the model say?

Same model, same prompt template, no matching chunk. Write down its first sentence.

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Running Code

► LIVE 4

Grounded prompt, two questions

```
in-corpus:  tyre pressure, car 27, wet  
out-of-corpus:  championship points of car 27
```

with grounding rule

without grounding rule

k = 4

lesson08_demo.ipynb

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Grounding: 'answer only from the context'

Context has the answer



Q: What tyre pressure for car 27 in the wet?

According to the wet-race procedure [handbook_procedures.md §4.2], car 27 runs 23.5 psi at the front and 21.0 psi at the rear, checked before release.

Context does not have the answer



Q: How many championship points does car 27 have?

The provided documents do not contain championship standings, so I cannot answer this from the team handbook.

Without the grounding rule the same model wrote: "Car 27 currently has 142 points, sitting fourth in the standings."

Load

Chunk

Embed

Store

Question

Retrieve

Augment

Generate

Indexing

Query time

Citing sources: metadata comes back

Indexing

```
chunk text  
+ vector  
+ {source, page, section}
```

Retrieval

```
top_k chunks arrive  
with their metadata  
attached
```

Prompt

```
[1] file $4.2 · text  
[2] file $2 · text
```



Trust

open it and check



Debugging

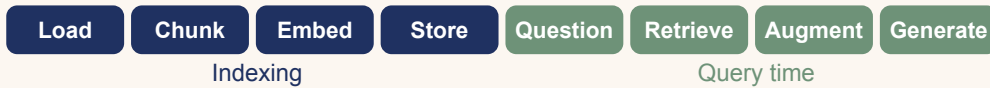
retrieval or generation?



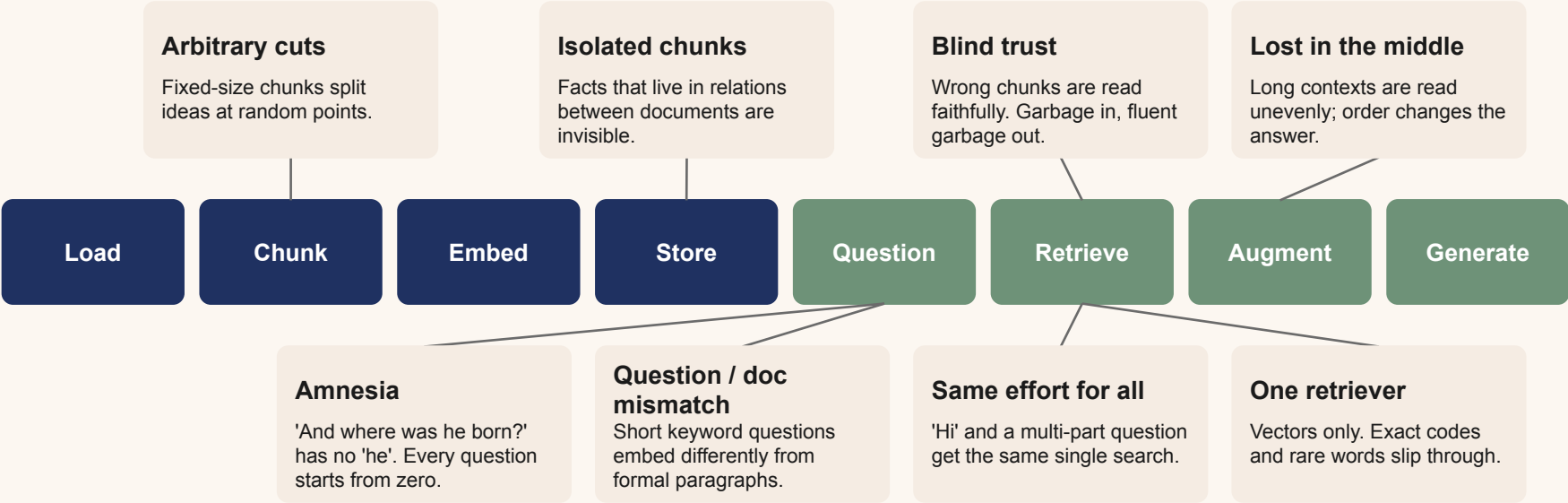
Filtering

only car 27, only after 2025

The payoff of the metadata we stored at indexing time.



Where vanilla RAG breaks



[Load](#)[Chunk](#)[Embed](#)[Store](#)[Question](#)[Retrieve](#)[Augment](#)[Generate](#)

Indexing

Query time

This week

Notebook · lesson08.ipynb

Ollama + nomic-embed-text + Chroma, Google Colab

- 1 Load the team handbook (fictional corpus, cars 27 and 88)
- 2 Chunk with fixed size and overlap
- 3 Embed and store in Chroma with metadata
- 4 Retrieve, inspect the top_k, compute recall@k
- 5 Generate with the grounded prompt and citations
- 6 Evaluate 20 questions, log results to CSV

Exercise

- 20 questions, with and without RAG, two Ollama models
- Vary chunk_size and top_k, report recall@k, accuracy, latency
- Two pages. Keep the CSVs for lesson 09